SAÉ 3.2 — Vérification et maintenance de la partie puissance et commande d'un système de conversion de l'énergie

Support de projet : robot auto-équilibré ELEGOO Tumblller

But générique : remise en service documentée, montée en version et coopération maître/suiveur

— Document étudiant : cahier des charges de projet —

BUT Génie Électrique et Informatique Industrielle

Département Génie Électrique et Informatique Industrielle

Volume horaire encadré : **24 h (8 séances de 3 h)** — Travail en groupes (un robot par groupe)

1 Cadre pédagogique

1.1 Place dans le cursus

Cette SAÉ correspond à la fiche nationale **SAÉ 3.EME.02 : « Vérification et maintenance de la partie puissance et commande d'un système de conversion de l'énergie »** du programme national du BUT GEII (arrêté du 15 avril 2022). Vous y serez placés dans une *situation professionnelle authentique* : un système existant vous est remis « sur étagère », avec son code d'origine, mais sans documentation de vérification ni de maintenance. Votre mission est double :

- remettre ce système *en service de manière documentée* (vérification de la partie puissance et de la partie commande, épuration du code) ;
- le faire *évoluer* : ajout d'une fonction de redressement autonome, puis mise en réseau de deux robots coopérant en mode maître/suiveur.

1.2 Compétences et apprentissages visés

Compétence	Niveau	Mise en œuvre dans ce projet
C2 — Vérifier la partie GEII d'un système	2	Élaborer et appliquer des **protocoles de test et de mesures** sur la chaîne de puissance (batterie, convertisseur, pont en H, moteurs) et la chaîne d'information ; interpréter les résultats ; identifier et analyser les dysfonctionnements.
C3 — Assurer le maintien en condition opérationnelle d'un système	1	Décrire et appliquer une **procédure de maintenance** ; remplacer ou ajouter une entrée/sortie et valider le fonctionnement ; **mettre à jour le programme** du système pour l'adapter aux nouvelles entrées/sorties (module ESP32).
C1 — Concevoir la partie GEII d'un système	2 (mobilisée)	Conception des fonctions nouvelles : stratégie de redressement autonome, protocole de communication maître/suiveur, machine à états de mission.
C4 — Installer tout ou partie d'un système de production, de conversion et de gestion d'énergie	1 (mobilisée)	Intégration matérielle du module ESP32, choix d'alimentation, câblage et mise en service de la liaison radio sur le système complet.
Compétences transversales	—	Gestion de projet (jalons, revues, dépôt Git), communication écrite (rapport final) et orale (soutenance / démonstration).

Remarque

Le rapport final sera rédigé de préférence en LaTeX (les modèles du département sont fournis) ; un traitement de texte classique reste toléré. La qualité de rédaction et de structuration du rapport fait partie de l'évaluation (section 8).

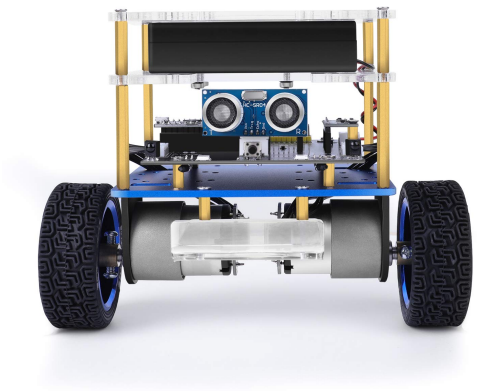
2 Consignes de sécurité

Avertissement sécurité — à lire avant toute manipulation

- **Accumulateur Li-ion 18650** : ne jamais le court-circuiter, ne jamais le percer ni le démonter ; ne charger que sur le chargeur fourni ; ne pas laisser la charge sans surveillance ; débrancher la batterie avant tout câblage ou modification du système.
- **Toujours couper l'alimentation** (batterie débranchée) avant de modifier le câblage, y compris lors de l'intégration des modules ESP32.
- Les **roues tournent** dès la mise d'équilibre : maintenir le robot hors sol lors des premiers essais après chaque téléversement ; protéger les doigts et les câbles.
- Le robot *tombe* : dégager la zone d'essais, faire les évolutions sur une surface dégagée et de niveau.
- Les modules ESP32 fonctionnent en **logique 3,3 V** alors que l'Arduino Nano est en logique 5 V : un pont diviseur ou un adaptateur de niveau est obligatoire sur la voie Nano → ESP32. Une entrée ESP32 soumise à 5 V peut être détruite.
- Précautions usuelles d'électricité et d'électronique : alimentations de laboratoire limitées en courant, multimètres vérifiés, sensibilité aux décharges électrostatiques.

3 Le système support : ELEGOO Tumblller

Le Tumblller est un robot **auto-équilibré sur deux roues** : il résout en permanence le problème du *pendule inverse* en mesurant son inclinaison (centrale inertielle) et en agissant sur la vitesse des deux moteurs, à cadence fixe. C'est, à petite échelle, une **chaîne de conversion d'énergie complète** : un accumulateur, un convertisseur, un étage de puissance (pont en H) et deux moteurs à réducteur, pilotés en largeur d'impulsion (PWM).



Le robot auto-équilibré ELEGOO Tumblller assemblé (photographie produit ELEGOO, reproduite à des fins pédagogiques).

3.1 Architecture matérielle (à confirmer par relevés lors de la réception)

Fonction	Élément	À relever ?
Traitement / commande	Arduino Nano (ATmega328P, 5 V, 16 MHz) sur carte d'extension ELEGOO	—
Mesure d'attitude	Module GY-521 : centrale inertielle MPU6050 (gyroscope + accéléromètre, bus I^2C)	—
Détection d'obstacles	Capteur à ultrasons (deux transducteurs, face avant) ; 2 capteurs infrarouge latéraux	portée utile
Puissance	Accumulateur Li-ion 18650 (3,7 V) ; convertisseur 5 V ; double pont en H pour les 2 moteurs CC à réducteur (référence du circuit à identifier sur la carte)	courant, rendement
Communication d'origine	Module Bluetooth basse consommation « BT16 » (application ELEGOO BLE Tool) ; récepteur infrarouge et télécommande	—
Interfaces	Bouton de démarrage, LEDs RVB, interrupteur d'alimentation	—

3.2 Chaîne d'énergie et chaîne d'information

Vous devrez produire, dès les premières séances, le **schéma fonctionnel** (chaîne d'information : acquérir, traiter, communiquer ; chaîne d'énergie : stocker, convertir, distribuer, transmettre) et le **schéma structurel** du système. Ces deux schémas sont des livrables de la SAÉ (cf. fiche nationale) et servent de référence à toute la suite du projet.

3.3 Le firmware d'origine

Le constructeur fournit le code source complet (voir annexe C). Sa structure est la suivante :

- une **interruption cadencée à 5 ms** (200 Hz) : lecture de la centrale inertielle, filtrage (filtre de Kalman), calcul des régulations d'équilibre (PD d'angle, PI de vitesse, PD de rotation) et génération des commandes PWM des moteurs ;
- la **boucle principale** : gestion du bouton, du module Bluetooth (application mobile), des capteurs (ultrasons, infrarouges), de la tension batterie, des animations de LEDs et des comportements autonomes (suivi, évitement, mode rebond).

Ce code est fonctionnel mais *monolithique* : tout est dans un seul fichier, avec des fonctions qui ne serviront pas à votre mission. Son épuration est l'objet de la partie 6.3.

4 Cahier des charges de la mission

4.1 Scénario professionnel

Votre équipe reçoit d'un client un robot Tumblller retiré du service, ainsi qu'un second exemplaire destiné à fonctionner en tandem. Le cahier des charges impose trois jalons.

4.2 Jalon 1 (séances 1 à 4) : remise en service documentée

1. **Réception et essais d'origine** : inventaire, contrôle visuel, mise sous tension, essais fonctionnels du firmware d'origine, procès-verbal de réception.

2. **Vérification de la partie puissance et commande** : mesures sur la chaîne d'énergie (tensions, courants, signaux PWM, autonomie), analyse et schémas.
3. **Récupération du projet et audit du code** : compilation du projet d'origine, cartographie du firmware, plan d'épuration.
4. **Épuration et refonte** : code modulaire, dépouillé des fonctions inutiles, comportement d'équilibre strictement conservé (tests de non-régression).

4.3 Jalon 2 (séance 5) : redressement autonome

Le robot, posé **couché sur le dos**, doit **se mettre debout tout seul** après un simple ordre (bouton ou message radio), puis repasser en mode équilibre. Aucune intervention mécanique n'est autorisée pendant la manœuvre.

4.4 Jalon 3 (séances 6 et 7) : coopération maître/suiveur

Deux robots communiquent par radio (modules ESP32 ajoutés, protocole ESP-NOW en Wi-Fi) :

— le robot **maître** roule et **détecte les obstacles** avec son capteur à ultrasons ; il **indique au second robot ce qu'il doit faire** (avancer, reculer, s'arrêter, tourner, se relever, cadencer le convoi) ;
— le robot **suiveur** exécute les ordres reçus tout en conservant son équilibre ;
— en cas de **perte de liaison**, chaque robot doit se mettre en **arrêt sécurisé** : la défaillance d'un équipement ne doit jamais créer un danger.

4.5 Exigences chiffrées (critères d'acceptation)

Réf.	Exigence	Valeur
EX1	Équilibre statique conservé après épuration du code (non-régression)	$\geq$ 60 s sans chute
EX2	Redressement autonome depuis la position couchée, sans aide	réussi en $\leq$ 15 s, $\leq$ 3 tentatives
EX3	Portée de la liaison maître/suiveur en salle de TP	$\geq$ 10 m
EX4	Latence moyenne entre décision du maître et action du suiveur	$\leq$ 200 ms
EX5	Perte de liaison $\Rightarrow$ arrêt sécurisé du suiveur	$\leq$ 500 ms
EX6	Intégrité des trames : toute trame invalide (CRC) est ignorée	0 action sur trame corrompue
EX7	Toute commande radio est tracée côté récepteur (journal d'essais)	100 % des trames

4.6 Scénario de la démonstration finale (séance 8)

1. Les deux robots sont couchés sur le sol ; le maître reçoit l'ordre de démarrage.
2. Le maître se redresse seul, puis ordonne au suiveur de se redresser à son tour (EX2).

3. Le suiveur démarre et suit le maître ; les deux robots roulent en convoi.
4. Un obstacle est placé devant le maître : détection par ultrasons, le maître s'arrête et ordonne l'arrêt au suiveur (EX4, EX6).
5. L'obstacle est retiré : reprise de la marche des deux robots.
6. L'enseignant coupe l'alimentation radio du maître : le suiveur doit se mettre en arrêt sécurisé (EX5) puis se stabiliser sans tomber.

5 Organisation du projet

5.1 Groupes et ressources

Un robot Tumblller et un module ESP32 par groupe (un second exemplaire du robot sera partagé entre deux groupes pour la partie jalon 3, planifié en binômes de groupes). Chaque groupe travaille avec un dépôt Git fourni et y versionne *tout* son code et ses documents.

5.2 Planning prévisionnel (24 h encadrées)

Séance	**Contenu**	**Jalon / revue**
S1 (3 h)	Réception, inventaire, essais du firmware d'origine	J1
S2 (3 h)	Mesures partie puissance et commande, schémas fonctionnel et structurel	J1
S3 (3 h)	Récupération du projet, compilation, audit et cartographie du code	J1
S4 (3 h)	Épuration, refonte modulaire, tests de non-régression	**Revue J1** : dossier de vérification initial + code épuré
S5 (3 h)	Fonction de redressement autonome	**Revue J2** : démonstration EX2
S6 (3 h)	Intégration ESP32, protocole de trames, essais de liaison ESP-NOW	J3
S7 (3 h)	Mission maître/suiveur, failsafe, scénario complet	**Revue J3** : démonstration EX3 à EX7
S8 (3 h)	Campagne de vérification finale, limites de fonctionnement, dossier de maintenance, soutenance	Démo finale + remise du rapport

Remarque

Chaque revue de jalon donne lieu à un point d'avancement de 10 minutes avec l'enseignant : ce qui est démontré, ce qui bloque, ce qui est replanifié. Les décisions sont consignées dans le dépôt Git (*commit* de revue). L'autonomie d'organisation du groupe fait partie de l'évaluation.

6 Déroulé détaillé

6.1 Séances 1 et 2 — Vérification du système livré (C2)

Principe : avant toute modification, vous devez *caractériser* le système tel qu'il vous est livré : c'est la base du dossier de vérification et de tout diagnostic ultérieur.

Procédure (séance 1) :

— Inventaire complet du colis (pièces, visserie, câbles) ; contrôle visuel de la carte d'extension, des soudures, des connecteurs ; procès-verbal de réception signé.
— Identification des composants de la carte (annexe B) : relever les références des circuits (microcontrôleur, pont en H, convertisseur) et compléter le tableau du système.
— Mise sous tension, téléversement du **firmware d'origine**, essais de toutes les fonctions usine : application Bluetooth, télécommande infrarouge, mode suivi, mode évitement d'obstacles, mode rebond, animations de LEDs.
— Consigner pour chaque essai : procédure, résultat attendu, résultat constaté (conforme / non conforme).

Procédure (séance 2) — mesures de la partie puissance et commande :

— Mesurer la tension batterie *au repos*, *sous charge* (robot en équilibre) et en *poussée maximale* (roues bloquées quelques instants, hors sol).
— Mesurer le courant total consommé dans les mêmes états (ampèremètre en série ou sonde) ; en déduire la puissance appelée dans chaque état.
— Relever à l'oscilloscope les signaux de commande PWM des moteurs et la tension aux bornes d'un moteur (avance, équilibre, oscillation volontaire) ; estimer la fréquence de découpage.
— Vérifier la tension du bus 5 V et son ondulation au repos et sous charge.
— Estimer l'autonomie énergétique : mesure de la décharge sur une durée d'essai donnée (30 min de fonctionnement modélisé), extrapolation, comparaison à l'usage réel.

Résultats à fournir

— Procès-verbal de réception (inventaire, anomalies, références des composants relevées).
— Schéma **fonctionnel** (chaînes d'information et d'énergie) et schéma **structurel** du système.
— Tableau de mesures puissance : (V_{bat}, I, P) pour chaque état, signaux PWM, ondulation du bus 5 V, courbe de décharge et estimation d'autonomie commentée.
— Procès-verbal d'essais des fonctions d'origine (tableau conforme / non conforme).

6.2 Séance 3 — Récupération du projet et audit du code

Principe : la mise à jour d'un programme existant commence par une *prise de connaissance méthodique* : on ne modifie jamais ce que l'on n'a pas compris.

Procédure :

— Récupérer le projet officiel du constructeur (annexe C), le compiler et le téléverser pour vérifier que vous reproduisez le comportement d'usine.
— **Cartographier le firmware** : pour chaque fonction du code d'origine, indiquer sa contribution au traitement (échantillonnage, filtrage, régulation, sortie PWM) ou aux interfaces

(Bluetooth, infrarouge, bouton, LEDs, capteurs, surveillance de tension). Localiser l'interruption 5 ms et identifier ce qui doit impérativement y rester.
- **Auditer** : lister les fonctions inutiles pour la mission (modes à supprimer), les valeurs numériques non nommées, les duplications, les chemins morts.
- Rédiger le **plan d'épuration** : ce que l'on garde, ce que l'on supprime, dans quel ordre, et les tests de non-régression à rejouer après chaque étape.

Résultats à fournir

Cartographie du code (tableau fonction → rôle → statut garder/supprimer) et plan d'épuration argumenté, validés en revue de fin de séance.

6.3 Séance 4 — Épuration et refonte du firmware

Principe : épurer ne veut pas dire « faire plus court » mais **ne conserver que ce qui sert la mission, de façon lisible, testable et vérifiable**. Le comportement d'équilibre d'origine doit être *strictement* conservé.

Procédure :
- Réorganiser le code en modules (par exemple : centrale inertielle, équilibre, moteurs, ultrasons, LEDs, communication, supervision), constantes nommées, en-têtes documentés.
- Supprimer par étapes successives les fonctions non retenues, en rejouant à chaque étape les **tests de non-régression** : équilibre statique 60 s (EX1), tenue après un choc léger, redressement par mode rebond d'origine.
- Versionner chaque étape dans Git ; toute régression doit pouvoir être annulée rapidement et l'incident consigné.
- Constitution du **dossier de vérification** : pour chaque test, objectif, procédure, matériel, résultat, conclusion.

Résultats à fournir

Firmware épuré (dépôt Git), journal des tests de non-régression, dossier de vérification initial conforme au modèle fourni.

6.4 Séance 5 — Redressement autonome (J2)

Principe : un robot à deux roues couché sur le dos est un pendule que l'on peut faire basculer en accumulant de l'énergie cinétique par un mouvement de *balancier* (aller-retours des roues synchronisés sur l'inclinaison mesurée). La détection du moment favorable au *verrouillage* de l'équilibre se fait sur la centrale inertielle.

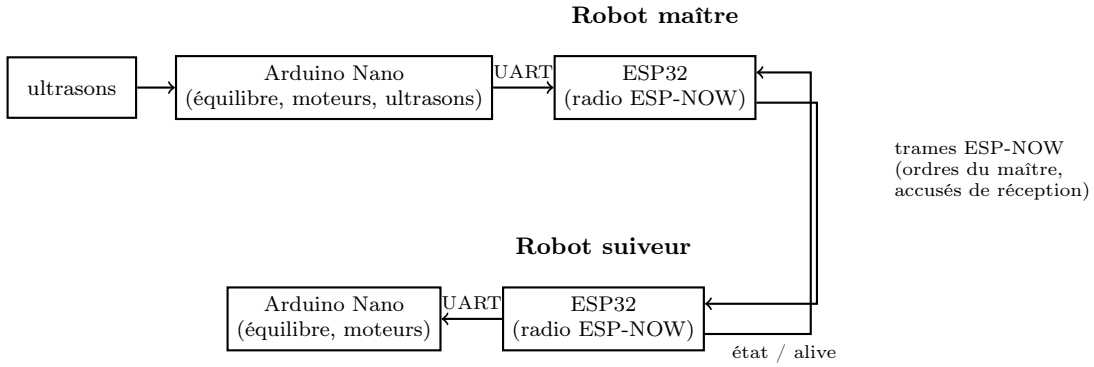
Procédure :
- Spécifier la machine à états : COUCHÉ → BALANCIER → TRANSITION → ÉQUILIBRE, avec critères de passage mesurés sur l'angle (et de repli en cas d'échec : nombre de tentatives borné).
- Implémenter en veillant à **ne jamais bloquer l'interruption 5 ms** : la manœuvre s'insère dans la boucle d'asservissement (le point de consigne d'angle et la vitesse peuvent y être modulés par le séquenceur).
- Régler le balancier expérimentalement (amplitude, fréquence, seuils) ; tracer l'angle mesuré pendant une manœuvre réussie et une manœuvre échouée (journal série).
- Tester sur plusieurs surfaces ; documenter les limites (moquette, pente, batterie partiellement déchargée).

Résultats à fournir

Démonstration EX2, chronogramme d'angle d'un relevage réussi, journal des essais de réglage et limites constatées.

6.5 Séance 6 — Intégration des ESP32 et liaison ESP-NOW

Principe : le module Bluetooth d'origine ne permet pas une liaison robot-robot simple ; on ajoute à chaque robot un module **ESP32** qui gère la radio (protocole ESP-NOW, liaison point à point sans passerelle Wi-Fi), l'Arduino Nano restant chargé de l'équilibre et des moteurs. Les deux processeurs dialoguent par **liaison série asynchrone**.



Procédure :

- Câbler chaque ESP32 : alimentation depuis le bus 5 V (broche `VIN`), masses communes, **adaptation de niveau 5 V → 3,3 V** sur la voie Nano → ESP32, liaison UART croisée RX/TX.
- Justifier le choix de l'UART matérielle et du débit ; expliquer le conflit UART/USB du Nano et la conséquence pour les messages de mise au point.
- Concevoir le **protocole d'échange** : trame compacte, octet de synchronisation, commande, somme de contrôle (CRC ou checksum) ; définir la table des commandes.
- Programmer les deux ESP32 (appariement par adresses MAC, envoi/réception ESP-NOW) ; valider d'abord *table à table* : échange de trames, puis mesure de portée (EX3), de latence (EX4) et de taux de perte en fonction de la distance.

Exemple de trame (à adapter) :

```
// Trame echangee Nano <-> ESP32 puis ESP-NOW (7 octets)
typedef struct __attribute__((packed)) {
  uint8_t  sync;    // octet de synchronisation (ex. 0xA5)
  uint8_t  cmd;     // STANDUP / MOVE_F / MOVE_B / STOP / TURN_L / TURN_R
      / ALIVE
  uint8_t  speed;   // consigne de vitesse (0..255)
  uint8_t  crc8;    // somme de controle des octets precedents
} trame_t;
```

Remarque

L'interruption d'équilibre (5 ms) doit rester *intouchable* : l'UART sera gérée dans la boucle principale, à cadence faible (par exemple 10 Hz côté maître), jamais dans l'urgence temps réel. Discuter dans le rapport le compromis débit / charge processeur retenu.

Résultats à fournir

Schéma de câblage Nano/ESP32 (avec adaptation de niveau), spécification du protocole (table des commandes + format de trame), courbes portée/latence/taux de perte, validation EX3, EX4 et EX6 sur banc.

6.6 Séance 7 — Mission coopérative maître/suiveur (J3)

Principe : le maître est le seul à *percevoir* (ultrasons) et à *décider* ; le suiveur *exécute*. La sûreté impose un mécanisme de *failsafe* : le suiveur ne roule que s'il reçoit régulièrement la preuve que le maître est actif.

Procédure :

— Côté maître : machine à états de mission (démarrage, convoi, obstacle, reprise, arrêt) fondée sur la distance mesurée par ultrasons ; émission périodique d'ordres et de messages de vie.

— Côté suiveur : exécution des ordres en conservant l'équilibre ; supervision du temps écoulé depuis le dernier ordre valide ; passe en **arrêt sécurisé** au-delà du seuil (EX5) :

```
// Boucle principale du suiveur (hors interruption 5 ms)
if (millis() - dernier_ordre_valide > 500) {
  mode = ARRET_SECURISE;   // perte de liaison -> stop
}
```

— Répéter le scénario complet de la démonstration finale (section 4.6) ; consigner chaque manœuvre dans le journal d'essais (EX7).

— Chaque groupe implémente les deux rôles (maître et suiveur) ; la démonstration est jouée en binômes de groupes lors de la séance 8.

Résultats à fournir

Code de mission (dépôt Git), journal des essais du scénario complet, validation EX5 à EX7.

6.7 Séance 8 — Vérification finale, limites, maintenance, soutenance

Procédure :

— **Campagne de vérification** : rejouer l'intégralité des tests des jalons 1 à 3 (procédures déjà rédigées) sur les robots en configuration finale.

— **Analyse des limites de fonctionnement** (dérisquage) : pente maximale, nature du sol, tension batterie minimale admissible, portée radio maximale, comportement en fin de course d'un capteur défaillant (par exemple ultrasons masqué).

— **Dossier de maintenance** : procédure de remplacement d'un moteur, d'un capteur et de la batterie ; procédure de mise à jour du firmware ; arbre de diagnostic d'une panne simple (le robot ne tient plus l'équilibre : quelle méthodologie ?).

— **Démonstration finale** devant jury (scénario section 4.6) et soutenance de 15 minutes (10 min de présentation, 5 min de questions) ; remise du rapport.

Résultats à fournir

Dossier de vérification final, analyse des limites de fonctionnement, dossier de maintenance, rapport final (préférablement LaTeX), soutenance.

7 Livrables (récapitulatif)

1. **Schéma fonctionnel et schéma structurel** du système.

2. **Dossier de vérification** : protocoles de test, résultats de mesures (partie puissance et commande), tests de non-régression, campagne finale.
3. **Dossier de maintenance** : procédures de maintenance et mise à jour du firmware, arbre de diagnostic.
4. **Analyse des limites de fonctionnement** (dérisquage).
5. **Code complet** (firmware épuré des deux rôles, code ESP32), versionné dans le dépôt Git du groupe, avec instructions de compilation.
6. **Rapport final** de projet (préférablement en LaTeX : structure, analyses, résultats, conclusions) et soutenance avec démonstration.

8 Évaluation

L'évaluation porte *à la fois* sur les livrables, les démonstrations et la démarche : chaque exigence EX1 à EX7 est démontrée en séance et tracée dans le dossier de vérification.

Réf.	Élément évalué	Points
V1	Réception, procès-verbal, essais d'origine, schémas fonctionnel et structurel	2
V2	Vérification partie puissance et commande (mesures, protocoles, autonomie)	2
V3	Audit et épuration du firmware (carte du code, refonte, non-régression EX1)	4
V4	Redressement autonome (EX2) : spécification, réglages, démonstration	3
V5	Liaison ESP-NOW : protocole, portée et latence (EX3, EX4, EX6)	3
V6	Mission maître/suiveur et failsafe (EX5, EX7), scénario complet	3
V7	Dossiers de vérification, de maintenance et analyse des limites	2
V8	Rapport final et soutenance (structure, clarté, remise à temps)	1
Total		**20**

Critères de réussite complémentaires : cohérence des unités et incertitudes dans les tableaux de mesures ; tout écart à une exigence doit être *analysé* (cause, criticité, action corrective) et non dissimulé ; le rapport en LaTeX (le cas échéant) suit la structure du modèle du département. La sécurité des manœuvres (batterie, câblage sous tension) est *rédhibitoire* : tout manquement grave entraîne la non-validation de la séance concernée.

Fin du sujet de projet

Annexe A — Matériel fourni par groupe

- 1 robot ELEGOO Tumblller monté (avec accumulateur Li-ion 18650 et chargeur) ;
- 1 module ESP32 (carte de développement, antenne intégrée) ;
- platine d'essai, fils de liaison, composants pour adaptation de niveau (résistances ou module convertisseur de niveau logique) ;
- multimètre, oscilloscope, alimentation de laboratoire réglable et limitée en courant ;
- poste informatique : Arduino IDE (ou PlatformIO), dépôt Git du groupe, modèle LaTeX du département ;
- accès au second robot (partagé entre deux groupes) pour le jalon 3.

Annexe B — Repères techniques du Tumblller

- Commande : Arduino Nano (ATmega328P), carte d'extension ELEGOO, alimentation par accumulateur Li-ion 18650 (3,7 V) et convertisseur 5 V.
- Mesure d'attitude : module GY-521 (MPU6050) sur bus I^2C ; échantillonnage à 200 Hz dans une interruption logicielle de 5 ms ; filtrage de Kalman ; régulations PD (angle), PI (vitesse), PD (rotation) ; commande PWM des deux moteurs CC à réducteur via double pont en H.
- Capteurs : ultrasons (face avant, distance typique de l'ordre de quelques cm à quelques dizaines de cm — à caractériser en séance 1), 2 détecteurs infrarouges latéraux, mesure de la tension batterie.
- Interfaces : bouton poussoir, LEDs RVB, module Bluetooth BT16 (application ELEGOO BLE Tool), récepteur infrarouge et télécommande.
- Code source officiel : dépôt du constructeur (annexe C) ; le projet d'origine tient en un seul fichier `.ino`.

Annexe C — Ressources documentaires

- Code et tutoriel officiels du constructeur : `https://github.com/elegooofficial/ELEGOO-TumbllerV1.1-Self-Balancing-Car-Tutorial`
- Manuel et fiches produit ELEGOO (carte fournie) : `https://www.elegoo.com/blogs/arduino-projects/elegoo-tumblller-self-balancing-robot-car-tutorial`
- Documentation ESP-NOW (Espressif) : `https://www.espressif.com/en/support/documents/technical-documents`
- Programme national BUT GEII (SAÉ 3.EME.02 et référentiel de compétences) — disponible sur l'intranet du département.

Fin du document étudiant